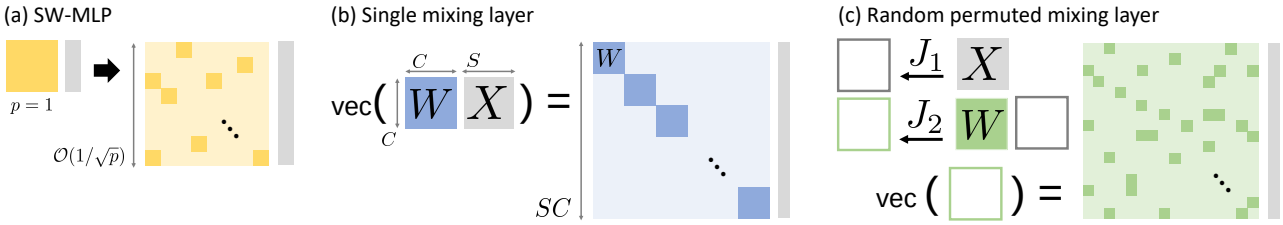

Figure 1. Schematic diagram of sparsity treated in this work. (a) A masked weight matrix $M \odot A$ in a sparse-weight MLP (SW-MLP). Its width is $O(1/\sqrt{p})$, where p is the ratio of non-zero entries in the mask M. (b) A mixing layer in an MLP-Mixer with the vectorization. The weight behaves as a block diagonal matrix. (c) A weight of a random permuted mixer (RP-Mixer), which is introduced in Section 5. The block diagonal structure is destroyed by random permutation matrices J_1, J_2 to achieve similarity to the SW-MLP.

The derivation is shown in Appendix B.4. It extends the fact that the Hadamard product parameterization has an implicit bias towards L1 regularization to the case of the Kronecker product parameterization (Hoff, 2017; Yasuda et al., 2023). Note that the number of independent parameters differs between the left and right sides of the inequality. Therefore, $\tilde{\lambda}$ might appear small, but it merely normalizes the change in parameter size (see Appendix B.4 for more details). The implicit bias towards sparsity is considered a desirable property in modern neural network architectures (Neyshabur et al., 2014; Woodworth et al., 2020).

3.3. Monarch matrix hidden behind Mixers

Dao et al. (2022) proposed a *Monarch matrix* $M \in \mathbb{R}^{n \times n}$ defined by

$$M = J_c^\top L J_c R, \tag{12}$$

where L and R are the trainable block diagonal matrices, each with $\sqrt{n}$ blocks of size $\sqrt{n} \times \sqrt{n}$. The previous work claimed that the Monarch matrix is sparse in that the number of trainable parameters is much smaller than in a dense $n \times n$ matrix. Despite this sparsity, by replacing the dense matrix with a Monarch matrix, it was found that various architectures can achieve almost comparable performance while succeeding in shortening the training time. Furthermore, the product of a few Monarch matrices can represent many commonly used structured matrices such as convolutions and Fourier transformations.

Surprisingly, the MLP Mixer and the Monarch matrix, two completely different concepts, have hidden connections. By comparing (10) and (12), we find that

Corollary 3.4. *Consider a S-Mixer without an intermediate activation function, that is, its feature matrix is given by $H = \phi(WXV)$. Then, vec(H) is equivalent to an MLP whose weight matrix is given by a Monarch matrix with weight-sharing diagonal matrices, that is, $\phi(Mx)$ with $n = SC$, $L = I_S \otimes V^\top$ and $R = I_C \otimes W$.*

We validate the similarities between the Monarch matrix and the Mixer through experiments. Here, we consider a shallow MLP ($x \mapsto B_c B_2 \mathrm{ReLU}(B_1 x)$) where B_c is the classification layer. We compare the performance of models where weights B_i ($i = 1, 2$) are replaced with Monarch matrices M_i, and models where they are replaced with Kronecker products $V_i \otimes W_i$. In Figure 2 (left), the two models showed comparable performance. This suggests that although the Mixer incorporates a weight-sharing structure compared to Monarch, its performance is not compromised.

3.4. Comparing hidden features

Let us back to the situation in practice where the model has intermediate non-linear activation. To investigate the similarity of non-linear networks, here we consider an *unstructured sparse-weight MLP* (SW-MLP in short), which is a basic implementation in the literature of naive MLPs with sparse weights (Golubeva et al., 2021). Its weights are given by a random sparse mask, that is, $M \odot A$ where A is the original weight matrix and M is a static mask matrix whose entries are drawn from the Bernoulli distribution with a probability $p > 0$ of being one at the initialization phase. In this section, to compare the MLP-Mixer with SW-MLP sharing conditions, we consider the average $p = (S^{-1} + C^{-1})/2$ of the sparsity of MLP-Mixer for the setting of SW-MLP. Figure 1 overviews the models that we compare in the below. The random permuted one is an alternative to SW-MLP and shows the result in Section 5.

As a similarity measure, we use the centered kernel alignment (CKA) (Nguyen et al., 2021) between hidden features of MLPs with sparse weights and those of MLP-Mixers. In practice, we computed the mini-batch CKA (Nguyen et al., 2021, Section 3.1(2)) among features of trained networks. In Figure 2(a), we observed the averaged CKA achieved the maximum as an appropriate sparsity of the MLPs. By comparing Figure 2(b) and (c), we found that CKA matrix with sparse MLP was clearly higher than dense MLP. In particular, the sparse Mixer was similar to sparser MLP in hidden features. Detailed settings of all experiments are summarized in Appendix C.

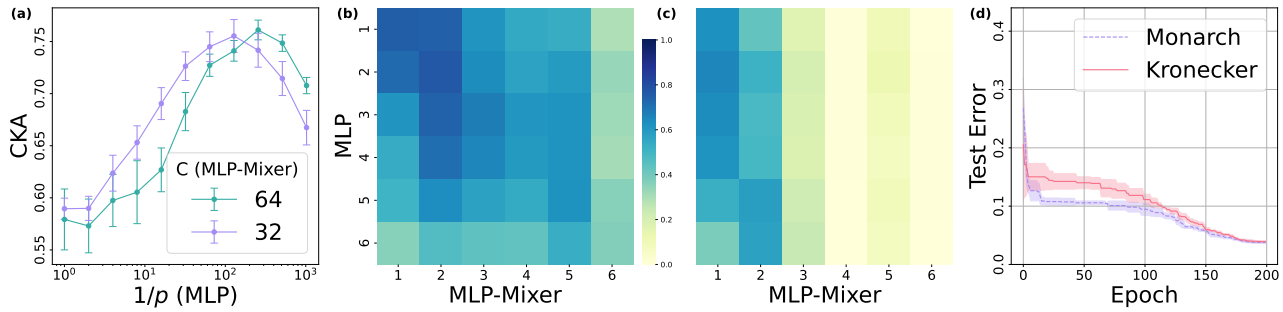

Figure 2. (a) Average of diagonal entries of CKA between trained MLP-Mixer ($S = C = 64, 32$) and MLP with different sparsity, where p is the ratio of non-zero entries in M. (b) CKA between MLP-Mixer ($S = C = 64$) and MLP with the corresponding $p = 1/64$, and (c) CKA between the Mixer and a dense MLP. (d) Test error on MNIST of shallow MLPs with Monarch matrix weights and Kronecker weights. The result is the average of five trials with different random seeds.

4. Properties as a wide MLP

In this section, we continue the discussion on whether the MLP-Mixer (or S-Mixer) exhibits tendencies similar to those of sparse-weight MLPs. In particular, we search for desirable sparsity and discuss whether there is a desirable setting of hyper-parameters on the Mixers.

4.1. Maximizing sparseness

The following hypothesis has a fundamental role:

Hypothesis 4.1 (Golubeva et al. (2021))**.** *Increasing the width up to a certain point, while keeping the number of weight parameters fixed, results in improved test accuracy.*

Intuitively, Golubeva et al. (2021) challenged the question of whether the performance improvement of large-scale deep neural networks was due to an increase in the number of parameters or an increase in width. They empirically succeeded in verifying Hypothesis 4.1; that is, the improvement was due to the increase in width in normal MLPs and ResNets (note that the width of ResNet indicates the channel size).

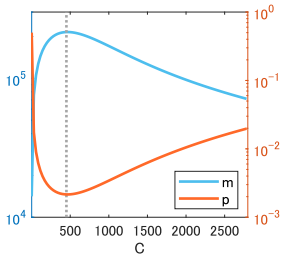

Figure 3. Theoretical line of m and p ($\Omega = 10^8, \gamma = 1$).

Let us denote by Ω the average number of connections per layer. We have

$$\Omega = p\gamma m^2, \tag{13}$$

where p is the ratio of non-zero entries in the static mask. Here, for the MLP-Mixer,

$$\Omega = \gamma(CS^2 + C^2S)/2. \tag{14}$$

The average number Ω of S-Mixer is reduced to $\gamma = 1$ in (14), which maintains the readability of the equations.

By (14), we have $S = (\sqrt{C^2 + 8\Omega/(\gamma C)} - C)/2$. For a fixed Ω and γ, the effective width is controlled by $m = SC$. Figure 3 shows m as a function of C. The width m has a single-peak form and is maximized at (C^*, S^*) as follows:

$$C^* = S^* = (\Omega/\gamma)^{1/3}, \quad \max_{S,C} m = (\Omega/\gamma)^{2/3}. \tag{15}$$

The ratio of non-zero entries $p = \Omega/\gamma m^2$ is minimized at this point, that is, the sparsity is maximized.

4.2. Comparing accuracy with effective width

To validate the similarity, we compare the test error of both networks with different sparsity. Under the fixed number Ω of connectivity per layer, the sparsity is equivalent to the wideness. Figure 4 (left) shows the test errors of MLP-Mixers and corresponding sparse weight MLPs under fixed $\Omega = 2^{19}$ and $\gamma = 2$, for several widths γm. We observed both networks' test error improved as the width increased. In this sense, MLP and MLP-Mixer have a similar tendency for performance with increasing width. However, we observed for too-wide cases around $\gamma m = 8000$ in Figure 4 (left), the test error of SW-MLP is higher than MLP-Mixer and there is little change in response to increasing width. We discuss this tendency in the next section.

4.3. Spectral analysis with increasing width

Golubeva et al. (2021) reported that if the sparsity became too high, the generalization performance of SW-MLP slightly decreased. They discussed that this decrease was caused by the deterioration of trainability, that is, it became difficult for the gradient descent to decrease the loss function. Some previous work reported that the large singular values of weight matrices at random initialization cause the deterioration of trainability in deep networks (Bjorck et al.,

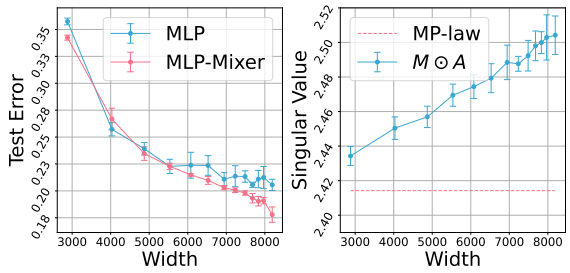

Figure 4. (left) Test error of MLPs with sparse weights and MLP-Mixers with different widths γm under the fixed Ω. We set $\Omega = 2^{19}$, $S = C = (\Omega/\gamma)^{1/3}$, and $\gamma = 2$. The x-axis represents the effective width γm. (right) The blue line indicates the averaged singular values of the weight $M \odot A$ of SW-MLP over five trials with different random seeds. The red line indicates c_γ, which is the square root of the right edge of the MP-Law.

2018). Therefore, we discuss the difference of singular values of weights between MLP-Mixer and SW-MLP.

In the case of Mixers, each $\gamma C \times C$ weight V (resp. $\gamma S \times S$ weight W) is initialized by i.i.d. random variables distributed with $\mathcal{N}(0, 1/C)$ (resp. $\mathcal{N}(0, 1/S)$). Then, by the theory of Marchenko-Pastur law (Bai & Silverstein, 2010), the weight's maximal singular value is approximated by $c_\gamma := 1 + \sqrt{\gamma}$. Since the Kronecker product with identity matrix does not change the maximal singular value, each maximal singular value of $V^\top \otimes I_S$ or $I_C \otimes W$ is approximated by c_γ.

Consider the case of SW-MLP. We initialize the entries of each mask matrix M by i.i.d. Bernoulli random variables with the probability of being $1/\sqrt{p}$ is p and being 0 is $1-p$. We initialize each weight A by i.i.d. $\mathcal{N}(0, 1/m)$. Consider the maximal singular value $\lambda_{\max}$ of $M \odot A$. Set $q = \sqrt{pm}$. Then by (Hwang et al., 2019, Theorem 2.9), $\lambda_{\max}$ is approximated by $\sqrt{L_+}$ in the following sense:

$$|\lambda_{\max}^2 - L_+| \prec 1/q^4 + 1/m^{2/3}, \tag{16}$$

where $\prec$ represents stochastic domination (Hwang et al., 2019, Definition 2.3), and

$$L_+ = c_\gamma^2 + 3c_\gamma^2\sqrt{\gamma}(1-p)/q^2 + O(1/q^4). \tag{17}$$

Under the fixed Ω, by (13), the dominant term of L_+ in (17) is linear in m as follows:

$$(1-p)/q^2 = \gamma m/\Omega - 1/m = O(m) \text{ as } m \to \infty. \tag{18}$$

Therefore, the maximal singular value $\lambda_{\max}$ of $M \odot A$ increases as the width increases. In Appendix D.1, we discuss the spectrum in large Ω limit and show the same tendency on the $\lambda_{\max}$ as large m limit.

In Figure 4 (right), we observed the maximal singular value of $M \odot A$ increased by widening m with fixed Ω and γ. In particular, the value was always higher than the theoretical value of Mixer's singular values. Such large singular values deteriorate the performance of SW-MLP. Conversely, we can enlarge the width and the sparsity of the MLP-Mixer without an undesirable increase in the maximal singular value.

5. Beyond the naive MLP on the width

As seen in Section 4.2, MLP-Mixers have similar tendencies to SW-MLPs. In much wider models, to continue comparing MLP-Mixer and unstructured sparse-weight MLP, we need an alternative to static-masked MLP because of its huge computational costs, memory requirements (Appendix D.9), and ill behavior on the spectrum (Section 4.3). Thus we further discuss partially destroying MLP-Mixer's structure and propose an alternative model of SW-MLP, which is called random permuted mixer (RP-Mixer).

5.1. PK family

To introduce an alternative to sparse-weight MLPs, we propose a permuted Kronecker (PK) family as a generalization of the MLP-Mixer.

Permutation matrix: An $m \times m$ permutation matrix J is a matrix given by $(Jx)_i = x_{\sigma(i)} (i = 1, 2, \ldots, m)$ for an index permutation σ. In particular, the commutation matrix J_c is a permutation matrix (Magnus & Neudecker, 2019). For any permutation matrix J, $x \in \mathbb{R}^m$, $J\phi(x) = \phi(Jx)$.

Definition 5.1 (PK layer and PK family)**.** Let J_1, J_2 be $m \times m$ permutation matrices. For $X \in \mathbb{R}^{n_1 \times n_2}$, we define the PK layer as follows:

$$\text{PK-Layer}_W(X; J_1, J_2) := \phi[J_2(I_{n_1} \otimes W)J_1\text{vec}(X)],$$

where we set $m = n_1 n_2$, $W \in \mathbb{R}^{n_2 \times n_2}$. We refer to the set of architectures whose hidden layers are composed of PK layers as the PK family.

Since J_c is a permutation matrix, the normal S-Mixer and MLP-Mixer belong to the PK family (See Section B.3 for the details). The important point of the PK-Layer is that its width m is possibly large, but there is *no need to explicitly preserve the $m \times m$ weight matrix* in memory. We can compute the forward signal propagation by a relatively small matrix multiplication in the same manner as the MLP-Mixer: First, $J_1\text{vec}(X) =: y$ is a rearrangement of X entries. Next, we compute pre-activation by using the matrix product $(I_{n_1} \otimes W)y = W\text{mat}(y)$. Finally, we apply entry-wise activation and rearrangement by J_2. Thus, the PK layer is memory-friendly, whereas the naive dense MLP requires preserving an $m \times m$ weight matrix and is computationally demanding.